

“小米杯”技术报告

目录

写在开头：我们是去年的赛博汪汪队，本次比赛沿用上次比赛的代码并作调整。	2
1 方案概述	2
1.1 lcm 通信接口	2
1.2 ros2 节点	2
2 创新点/亮点概述	3
3 关键技术	4
3.1 环境感知	4
3.2 导航及路径规划	6
3.3 运动控制	9
3.4 特殊地形处理	11

写在开头：我们是去年的赛博汪汪队，本次比赛沿用上次比赛的代码并作调整。

1 方案概述

1.1 lcm 通信接口

使用到的 lcm 通信接口有：

- `robot_control_cmd` 用来控制机器狗的运动，在方案实现中通过控制 `mode` 和 `gait` 来完成不同的步态，通过设置 `x`、`y`、`z` 轴的速度和身体高度等参数来完成相对应的动作。
- `robot_control_response` 用来记录机器狗的自身状态，我们通过订阅该节点，查看了目前机器狗动作的切换状态(`switch_status`)和动作的执行进度(`order_process_bar`)，并检测机器狗是否有报错。当机器狗处于趴下状态时，会更新指令，使机器狗站立。
- `simulator_state` 用来记录机器狗环境的状态，我们使用了该节点返回的 `x`、`y`、`z` 位置信息，记录机器狗的运动位置，便于在不同位置执行不同动作；使用该节点的 `rpy` 信息，并将航偏角 `rpy[2]` 从 180° 的范围扩充到 360° ，便于进行方位判断。
- `user_gait_file` 用于自定义步态的发布，我们通过该 lcm 将自定义步态的定义和全部参数发布出去，后续只需使用 `mode=62`，`gait=110` 即可使用该步态，我们所实现的蹄形步态（3.3.1）就是使用的该 lcm 发布。

1.2 ros2 节点

使用到的 ros2 节点有：

- `/rgb_camera/image_raw`，这是添加的 rgb 相机，返回结果为相机拍摄照片的 3 颜色通道图片。

2 创新点/亮点概述

- 通过 rpy(滚转、俯仰、偏航角)得出机器狗实际的位置朝向，在 simulator_state 节点返回的 rpy 参数中，yaw 航偏角实际范围为 0~180，无法分辨出正前方和正后方的位置角度，我们通过 roll 滚转角接近 0 或者接近 180 来判断机器狗在正面或者在后面，当 roll 接近 0 度时，我们认为机器狗在正面，yaw 角度即为原来的角度；当 roll 接近 180 度时，我们认为机器狗在背面，yaw 角度为原来角度加上 180 度，成功将角度信息从 180 度扩展到了 360 度。
- 设计蹄行步态，通过减少每次迈步的抬腿数量，增加稳定性，并降低身体姿态，能够更稳定的通过限高杆路段。
- S 弯动态调整转弯速度和机体位置，避免踩线。
- 使用 opencv 处理 rgb 相机返回的图像，识别二维码、箭头方向、黄灯。

3 关键技术

3.1 环境感知

3.1.1 环境感知介绍

- 在需要相机检测的部分，使用 rgb 相机获取图像。
 - 通过获取 rgb 相机传回的图像，对二维码内容、箭头方向、黄灯距离进行识别计算，根据得到的结果做出不同的响应。
- 在斜坡关卡，使用了机器狗的地形检测模块。
 - 通过地形检测机器狗可以修改自身的运动轨迹以及受力，来完成上下斜坡的动作。
- 在赛道中，判断机器狗的位置。
 - 通过不断检测机器狗的位置，来判定机器狗目前处于那一关卡，并根据不同的关卡设计对应的动作，使机器狗尽可能维持在赛道的中间位置，避免碰撞。

3.1.2 RGB 相机添加

- 修改 gazebo.xacro 文件，具体增加代码如下：

```

</gazebo>
<gazebo reference="RGB_camera_link">
  <sensor type="camera" name="rgb_camera">
    <always_on>true</always_on>
    <update_rate>15.0</update_rate>
    <camera name="rgb_camera">
      <horizontal_fov>1.46608</horizontal_fov>
      <image>
        <width>2560</width>
        <height>1440</height>
        <format>R8G8B8</format>
      </image>
      <distortion>
        <k1>0.0</k1>
        <k2>0.0</k2>
        <k3>0.0</k3>
        <p1>0.0</p1>
        <p2>0.0</p2>
        <center>0.5 0.5</center>
      </distortion>
    </camera>
    <plugin name="rgb_camera_plugin" filename="libgazebo_ros_camera.so">
      <ros>
        <!--<namespace>stereo</namespace>-->
        <remapping>~/image_raw:=image_raw</remapping>
        <remapping>~/camera_info:=camera_info</remapping>
      </ros>
      <!--Set camera name. If empty, defaults to sensor name (i.e. "sensor_name")-->
      <camera_name>rgb_camera</camera_name>
      <!--Set TF frame name. If empty, defaults to link name (i.e. "link_name")-->
      <frame_name>RGB_camera_link</frame_name>
      <hack_baseline>0.2</hack_baseline>
    </plugin>
  </sensor>
</gazebo>

```

3.2 导航及路径规划

3.2.1 基础控制逻辑

通过检测机器狗的位姿信息，控制机器狗执行不同的步态。

```
with data_lock:
    num = select_step_based_on_position(pos_msg.position, gait_msg.gait_mode, pos_msg.rpy
```

通过 flags、results 两个状态数组判断机器狗处于哪一段赛道上。（这里只展示部分 flags）

```
20 flags = {
21     "ENDING_FLAG1" : False, # 起点前进标志
22     "ENDING_FLAG2" : False, # A区配货标志
23     "ENDING_FLAG3" : False, # A->B过S弯标志
24     "ENDING_FLAG4" : False, # 前往B区标志
25     "ENDING_FLAG5" : False, # B区卸货装货标志
26     "ENDING_FLAG6" : False, # B区完成回返A区标志
27     "ENDING_FLAG7" : False, # B->A过S弯标志
28     "ENDING_FLAG8" : False, # 卸货前进入A区标志
29     "ENDING_FLAG9" : False, # A区卸货标志
30     "ENDING_FLAG10" : False, # 结束标志
31     # A区装货标志
32     "A_LOAD_ENDING_FLAG1" : False,
33     "A_LOAD_ENDING_FLAG2" : False,
34     "A_LOAD_ENDING_FLAG3" : False,
35     "A_LOAD_ENDING_FLAG4" : False,
36     "A_LOAD_ENDING_FLAG5" : False,
37     # A区卸货标志
38     "A_UNLOAD_ENDING_FLAG1" : False,
39     "A_UNLOAD_ENDING_FLAG2" : False,
40     "A_UNLOAD_ENDING_FLAG3" : False,
41     "A_UNLOAD_ENDING_FLAG4" : False,
42     "A_UNLOAD_ENDING_FLAG5" : False,
43     "A_UNLOAD_ENDING_FLAG6" : False,
44     "A_UNLOAD_ENDING_FLAG7" : False,
```

```

results = {
    "A_QR" : 0, # A区二维码识别结果（装货库位）    A-1: 1    A-2: 2
    "B_QR" : 0, # B区二维码识别结果（卸货库位）    B-1: 3    B-2: 4
    "ARROW" : 0, # 箭头识别结果    朝左: 1        朝右: 2
    "yellow_light": 0, # 黄灯识别标志        未识别: 0    已识别: 1
    "ready_wait" : 0, # 黄灯停止标志        未等待: 0    已等待: 1
    "ready_load_a" : 0, # A区装货标志        未装货: 0    已装货: 1
    "ready_unload_a" : 0, # A区卸货标志        未卸货: 0    已卸货: 1
}

```

通过检测机器狗的状态信息，控制机器狗恢复站立。

```

def select_step_based_on_position(position, gait_mode, rpy):
    global flags, results
    x, y, z = position
    gait, mode = gait_mode
    rpy=rpy
    if gait==0 and mode==0 or gait ==1 and mode==9:
        return 0

```

3.2.2 A 库位赛段

在起点位置，y 轴为赛道宽度，x 轴正方向为机器狗前进方向。首先向前移动，在转弯处检测当前的偏航角进行旋转，将机器狗的前进方向旋转至 y 轴负方向。旋转完成后向前移动识别二维码，根据识别的二维码内容前往对应的库位进行装货（本项目设计的是 A-2 库位），到达库位后趴下完成装货，再从库位返回 S 弯的起点处。

3.2.3 S 弯赛段

通过第一个转弯后，y 轴为赛道宽度，x 轴正方向为机器狗前进方向。机器狗向前移动，选择慢走步态并不断调整机身位置。进入 S 弯之后，根据机器狗的位置确定机器狗所处 S 弯赛段，选择对应的左转弯或右转弯步态，并根据机器狗与弯道圆心的距离和偏离角度实时修正机器狗的偏航角和位置，保证机器狗始终处于赛道中央，不会踩踏黄线。

3.2.4 箭头识别

在出 S 弯之前，识别箭头，识别到了之后进入对应的赛段中，本项目设计的是向右走。

3.2.5 上下斜坡赛段

通过 S 后，首先进行连续转两个弯，进行斜坡前准备，控制机器狗的前进方向为 y 轴正方向，调整完毕，边行走边调整执行慢走步态进行爬坡和下坡。

3.2.6 黄灯赛段

通过斜坡后，先停下来相机检测黄灯位置，计算出黄灯距离，然后走到规定距离内停下等待黄灯。

3.2.7 B 库位赛段

通过黄灯过后边走边进行二维码库位识别，当识别到一次后，机器狗根据判断语句走到对应的库位中卸货，并走到另一个库中装货。

3.2.8 限高杆赛段

边走边判断 y 的位置，当进入到限高杆区域后，开始执行自定义步态，压低身体姿势并采用蹄行步态稳定的通过赛段。

3.2.9 石板赛段

检测 y，当快要进入石板赛段时，开始抬高抬腿高度，降低行进速度，并边走边根据 x 的位置进行左右调整，以便稳定的通过赛段。

3.2.10 S 弯返回赛段

完成转弯后，机器狗进入 S 弯返回赛段，根据机器狗的位置确定机器狗所处 S 弯赛段，选择对应的左转弯或右转弯步态，并根据机器狗与弯道圆心的距离和偏离角度实时修正机器狗的偏航角和位置，保证机器狗始终处于赛道中央，不会踩踏黄线。

3.2.11 A 库位返程

完成 S 弯返程后，根据储存的 A 区二维码结果，走到另外一个库位进行卸货，卸货完成后前往起点完成比赛。

3.3 运动控制

3.3.1 蹄行步态：限高杆

基本介绍：蹄行步态是为了度过限高杆而设计的一种稳定性很高的自定义步态，在整个步态中四只脚中至少有三只脚都保持接触地面，只有一只脚在移动，同时降低身体高度，保证不与限高杆发生碰撞。

步态格式：依靠运动文档中给出的自定义步态格式写出，分为定义和参数两个部分。

定义：

```
[[section]]
```

```
contact = [1, 1, 1, 1]
```

```
duration = 3
```

参数：

```
# 1
```

```
[[step]]
```

```
type = "usergait"
```

```
body_vel_des = [0.1, 0.0, 0.0]
```

```
body_pos_des = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

```
landing_pos_des =
```

```
[0.065, 0.00, 0.0, 0.00, 0.00, 0.0, 0.00, 0.00, 0.0, 0.00, 0.00, 0.00, 0.00]
```

```
step_height = [0.13, 0.13, 0.13, 0.13]
```

```
weight = [10.0, 10.0, 10.0, 50.0, 50.0, 10.0]
```

```
use_mpc_traj = 0
```

```
mu = 0.40
```

```
landing_gain = 1.0
```

```
gait_id = 110
```

```
duration = 270
```

实现方法：始终保持四只脚中至少有三只脚都保持接触地面，每次抬起一只脚前进，然后放下，遵循 FL、RR、FR、RL 的顺序执行，并设定一定身体姿态，保证不会被碰到限高杆。

3.4 特殊地形处理

3.4.1 相机部分

定义一个公用的图像接收类 ImageHandler

这个类拥有三个成员：

bridge: 图片类型转化器，将从 msg 中接受的图像转化为 opencv 能处理的类型。

image_received: 图片接收标志位，用于标志当前是否已经接收到可用图像。

current_image: 当前接收到的图片，用于储存转化后的图像。

这个类拥有一个回调处理函数：

image_callback: 使用图片类型转化器将从 msg 中接收到的图像转化为 “bgr8” 格式，储存在 current_image 中并把图片接收标志位置为 True。

```
class ImageHandler:
    def __init__(self):
        self.bridge = CvBridge()
        self.image_received = False
        self.current_image = None

    def image_callback(self, msg):
        self.current_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        self.image_received = True
```

3.4.2 二维码识别：

首先实例化一个图像接收类的对象，再初始化 ROS 节点。为 ROS 节点设置 QoS 策略，注意这里 reliability 必须和 gazebo 的 ROS 节点的一致，采用 BEST_EFFORT。

```
def Qrcode():
    """
    识别二维码

    returns:
        int:
            A-1返回1
            A-2返回2
            B-1返回3
            B-2返回4
    """

    # 创建图像处理程序并设置ROS订阅
    image_handler = ImageHandler()

    # 初始化ROS节点
    rclpy.init(args=None)
    node = rclpy.create_node('image_listener')
    qos_profile = QoSProfile(
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        history=QoSHistoryPolicy.KEEP_LAST,
        depth=10
    )
```

为这个节点创建一个订阅，从 RGB 相机发布的 image_raw 主题中获取图像，图像类型为 image 类，调用图像接收处理器的回调函数对图像进行处理。

```
subscription = node.create_subscription(
    Image,
    '/rgb_camera/image_raw',
    image_handler.image_callback,
    qos_profile
)
```

接收到有效图像之后，使用 ROI 策略，因为二维码的图像永远会在图像的上半部分，所以直接截取图像的上半部分。

```
# 等待接收第一张图像
while not image_handler.image_received:
    rclpy.spin_once(node)
image = image_handler.current_image

# 截取上半部分
height, width = image.shape[:2]
upper_half = image[0:height//2, 0:width]
```

将截取后的图像进行灰度处理，再进行直方图均衡，最后二值化，消除环境对二维码识别的影响。

```
gray = cv2.cvtColor(upper_half, cv2.COLOR_BGR2GRAY)
equal = cv2.equalizeHist(gray)
thresh = cv2.threshold(equal, 1, 255, cv2.THRESH_BINARY)[1]
```

处理后的图像使用 opencv 的 decode 库进行处理，识别出二维码的内容，根据二维码的内容返回不同的值。

```

decoded = decode(thresh)
result = 0
if decoded:
    qr_data = decoded[0].data.decode('utf-8')
    node.get_logger().info(f"识别到二维码: {qr_data}")
    if qr_data == "A-1":
        result = 1
        node.get_logger().info("识别结果: A-1 -> 返回 1")
    elif qr_data == "A-2":
        result = 2
        node.get_logger().info("识别结果: A-2 -> 返回 2")
    elif qr_data == "B-1":
        result = 3
        node.get_logger().info("识别结果: B-1 -> 返回 3")
    elif qr_data == "B-2":
        result = 4
        node.get_logger().info("识别结果: B-2 -> 返回 4")
    else:
        node.get_logger().warn(f"未知二维码: {qr_data}")
else:
    node.get_logger().warn("未能识别到二维码")

```

识别完成后关闭节点并清理资源。

```

# 清理资源
node.destroy_subscription(subscription)
rclpy.shutdown()
return result

```

3.4.3 箭头方向识别:

首先实例化一个图像接收类的对象，再初始化 ROS 节点。为 ROS 节点设置 QoS 策略，注意这里 reliability 必须和 gazebo 的 ROS 节点的一致，采用 BEST_EFFORT。

```
def arrow():
    # 创建图像处理程序并设置ROS订阅
    image_handler = ImageHandler()

    # 初始化ROS节点
    rclpy.init(args=None)
    node = rclpy.create_node('image_listener')
    qos_profile = QoSProfile(
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        history=QoSHistoryPolicy.KEEP_LAST,
        depth=10
    )
```

为这个节点创建一个订阅，从 RGB 相机发布的 image_raw 主题中获取图像，图像类型为 image 类，调用图像接收处理器的回调函数对图像进行处理。

```
subscription = node.create_subscription(
    Image,
    '/rgb_camera/image_raw',
    image_handler.image_callback,
    qos_profile
)
```

接收到有效图像之后，因为箭头是绿色的，将图像转化到 HSV 空间中，根据定义的绿色 HSV 范围，创建一个绿色掩膜，只留下图像中绿色的部分。

```
# 等待接收第一张图像
while not image_handler.image_received:
    rclpy.spin_once(node)
image = image_handler.current_image
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
# 定义绿色HSV范围
lower_green = np.array([35, 50, 50])
upper_green = np.array([85, 255, 255])

# 创建绿色掩膜
mask = cv2.inRange(hsv, lower_green, upper_green)
```


定义 5x5 的核心，使用开运算去除噪声，闭运算连接区域，得到更高质量的箭头图片。

```
# 创建绿色掩膜
mask = cv2.inRange(hsv, lower_green, upper_green)

# 开运算去除噪声，闭运算连接区域
kernel = np.ones((5,5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

对处理后的图像查找轮廓并过滤小轮廓，确保只留下箭头的轮廓。

```
# 查找轮廓
contours = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0]

# 过滤小轮廓
contours = [cnt for cnt in contours if cv2.contourArea(cnt) > 100]

if not contours:
    return 1# 默认

# 取最大轮廓（假设箭头是最大的绿色区域）
largest_contour = max(contours, key=cv2.contourArea)
```

得到箭头轮廓之后，使用两种方式判断箭头方向。首先尝试采用凸包和凸缺陷判断方向，如果没有明显凸包和凸缺陷，则采用外接矩形的方法判断箭头方向。根据箭头方向返回不同的值。


```

# 计算最小外接矩形
rect = cv2.minAreaRect(largest_contour)
box = cv2.boxPoints(rect)
box = np.int0(box)

# 计算轮廓的凸包和凸缺陷
hull = cv2.convexHull(largest_contour, returnPoints=False)
defects = cv2.convexityDefects(largest_contour, hull)
if defects is None or len(defects) < 3:
    # 如果没有明显凸缺陷, 使用外接矩形方向
    angle = rect[2]
    if angle < -45:
        angle += 90

    if -90 <= angle < 90:
        direction = "right"
    else:
        direction = "left"

```

```

else:
    # 使用凸缺陷方法确定箭头尖端
    farthest_point = None
    max_dist = 0

    for i in range(defects.shape[0]):
        f, d = defects[i, 0][2:4]
        far = tuple(largest_contour[f][0])

        if d > max_dist:
            max_dist = d
            farthest_point = far

    if farthest_point:
        # 计算质心
        M = cv2.moments(largest_contour)
        cx = int(M['m10']/M['m00'])

        # 计算方向向量
        dx = farthest_point[0] - cx
        direction = "right" if dx > 0 else "left"
    else:
        direction = "unknown"

node.get_logger().info(f"识别结果: 方向为{direction}")

```

识别完成后关闭节点并清理资源。

```
# 清理资源
node.destroy_subscription(subscription)
rclpy.shutdown()
return 1 if direction == "left" else 2
```

3.4.4 黄灯识别:

首先定义相机高度和黄灯高度，实例化一个图像接收类的对象，再初始化 ROS 节点。为 ROS 节点设置 QoS 策略，注意这里 reliability 必须和 gazebo 的 ROS 节点的一致，采用 BEST_EFFORT。

```
def yellow_light():
    """
    进行语音播报(5,4,3,2,1)

    returns:
        int: 播报完成返回1
    """

    camera_height = 0.3 # 相机高度
    light_height = 0.6 # 黄灯高度
    # 创建图像处理程序并设置ROS订阅
    image_handler = ImageHandler()

    # 初始化ROS节点
    rclpy.init(args=None)
    node = rclpy.create_node('image_listener')
    qos_profile = QoSProfile(
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        history=QoSHistoryPolicy.KEEP_LAST,
        depth=10
    )
```

为这个节点创建一个订阅，从 RGB 相机发布的 image_raw 主题中获取图像，图像类型为 image 类，调用图像接收处理器的回调函数对图像进行处理。

```
subscription = node.create_subscription(
    Image,
    '/rgb_camera/image_raw',
    image_handler.image_callback,
    qos_profile
)
```

接收到有效图像之后，使用 ROI 策略，因为黄灯永远在图像的上半部分，所以截取上半部分的图像以去除地面赛道的影响。因为黄灯是黄色的，将图像转化到 HSV 空间中，根据定义的黄色 HSV 范围，创建一个黄色掩膜，只留下图像中黄色的部分。

```
# 等待接收第一张图像
while not image_handler.image_received:
    rclpy.spin_once(node)
image = image_handler.current_image
height, width = image.shape[:2]
# 定义ROI（上半部分，去除地面影响）
upper_half = image[0:height//2, 0:width]
hsv = cv2.cvtColor(upper_half, cv2.COLOR_BGR2HSV)
# 定义黄色HSV范围
lower_yellow = np.array([20, 100, 100])
upper_yellow = np.array([30, 255, 255])

# 创建黄色掩膜
mask = cv2.inRange(hsv, lower_yellow, upper_yellow)
```

此时还有黄灯两边细杆的影响，定义 200x200 的核心，使用开运算去除噪声，闭运算连接区域，去除黄灯两边细杆。

```
# 开运算去除噪声，闭运算连接区域(这里去除连接杆的影响)
kernel = np.ones((200,200), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

对处理后的图像查找轮廓并过滤小轮廓，确保只留下黄灯轮廓。

```
# 查找轮廓
contours = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0]

# 过滤小轮廓
contours = [cnt for cnt in contours if cv2.contourArea(cnt) > 100]

if not contours:
    return 1# 默认

# 取最大轮廓
largest_contour = max(contours, key=cv2.contourArea)
```

得到黄灯轮廓之后，计算黄灯的中心点。相机的水平 FOV 大小、图像的长宽均已知，计算相机的垂直 FOV，再计算相机的垂直焦距。

```
# 计算黄灯中心点
bottom_height = max(largest_contour, key=lambda pt: pt[0][1])[0][1]

# 计算垂直FOV
horizontal_fov = np.deg2rad(84) # 84° (弧度)
aspect_ratio = width / height
vertical_fov = 2 * np.arctan(np.tan(horizontal_fov / 2) / aspect_ratio)

# 计算垂直焦距
focal_length_v = (height / 2) / np.tan(vertical_fov / 2)
```

再计算图像的中心点，通过相似三角形法，计算当前与黄灯的距离。

```
# 图像中心点
center_v = height // 2

# 距离计算
distance = (focal_length_v * (light_height - camera_height)) / (center_v - bottom_height)
node.get_logger().info(f"当前距离黄灯: {distance}m")
```

识别完成后关闭节点并清理资源。

```
# 清理资源
node.destroy_subscription(subscription)
rclpy.shutdown()
return distance
```


3.4.5 S 弯：

定义左转 S 弯步态 25 和右转 S 弯步态 26

其转弯的曲率根据 S 弯的半径计算而得

通过这两个基本的转弯步态，配合各个 S 弯分段之间的临界点判断，已经可以让机器狗完整通过 S 弯

为了保证在通过 S 弯的过程中机器狗始终保持在弯道中央，不会出现踩踏黄线的情况，需要对机器狗与弯道圆心的距离和偏差角度进行实时检测，并通过修正步态进行实时修正。

修正步态：

左转 S 弯急弯步态——20 在左转 S 弯步态的基础上，转弯曲率更大

左转 S 弯缓弯步态——21 在左转 S 弯步态的基础上，转弯曲率更小

右转 S 弯急弯步态——22 在右转 S 弯步态的基础上，转弯曲率更大

右转 S 弯缓弯步态——23 在右转 S 弯步态的基础上，转弯曲率更小

左转步态——2 原地左转

右转步态——3 原地右转

左移步态——7 保持朝向不变，向左移动

右移步态——8 保持朝向不变，向右移动

修正策略：

（1）首先是偏差距离的修正：

以去程的第一个 S 弯（左转过弯）为例，通过

```
distance = math.sqrt((x - 1.47)**2 + (y - 0.95)**2)
```

实时计算机器狗与 S 弯圆心的距离。

如果距离略小于半径，则需要使用左转 S 弯缓弯步态进行微调修正

如果距离略大于半径，则需要使用左转 S 弯急弯步态进行微调修正

如果距离小于半径超出阈值，则需要使用右移步态进行修正

如果距离大于半径超出阈值，则需要使用左移步态进行修正

```
if(distance<0.72 and distance>=0.68):  
    return 21  
elif(distance <0.68):  
    return 8  
elif(distance>0.74 and distance<=0.78 ):  
    return 20  
elif(distance>0.78):  
    return 7
```

(2) 首先是偏差角度的修正：

同样以去程的第一个 S 弯（左转过弯）为例，通过

```
target_rpy = math.degrees(math.atan2(y-0.95,x-1.47))+180  
target_rpy -= 90  
delta_rpy = rpy - target_rpy
```

实时计算机器狗在该位置的理想角度偏差

如果该角度值偏大，那么需要采用右转步态进行修正；如果该角度偏小，那么需要采用左转步态进行修正。

```
if(delta_rpy>5 and delta_rpy<80):  
    return 3  
elif(delta_rpy<-5 and delta_rpy>-80):  
    return 2
```

去程第一个 S 弯的完整代码为：（其余弯道同理）

```
# 过S弯，并在最后一个S弯识别箭头方向（朝右）
def pass_s_and_identify_arrow(position,rpy):
    global flags,results
    x, y, z = position
    rpy=rpy
    if x< 1.47 and flags["S_ENDING_FLAG1"] == False:
        return walk_0(rpy)
    elif 0.95 <x and flags["S_ENDING_FLAG2"] == False:
        flags["S_ENDING_FLAG1"] = True
        distance = math.sqrt((x - 1.47)**2 + (y - 0.95)**2)
        # print(distance)
        target_rpy = math.degrees(math.atan2(y-0.95,x-1.47))+180
        target_rpy -= 90
        delta_rpy = rpy - target_rpy
        # print(delta_rpy)
        if(delta_rpy>5 and delta_rpy<80):
            return 3
        elif(delta_rpy<-5 and delta_rpy>-80):
            return 2
        if(distance<0.72 and distance>=0.68):
            return 21
        elif(distance <0.68):
            return 8
        elif(distance>0.74 and distance<=0.78 ):
            return 20
        elif(distance>0.78):
            return 7
        return 25
```

通过上述策略，可以实现机器狗在完整通过 S 弯的同时，实时微调，保证自身始终处于道路中央，不会踩踏黄线。

3.4.6 限高杆

自定义步态，采用蹄形步态降低姿势，提高稳定性，并不碰撞限高杆。

3.4.7 石板赛段

提高抬腿高度，边走边调整，稳定通过石板赛段。